Interactive Vision-Assisted Object Grasping for the Visually Impaired

This is the GitHub Link

1. Project Overview

This project is an assistive vision-language system designed to help a visually impaired user locate and grasp a target drink, which connects real-time camera input, speech interaction, vision-language inference, and spoken feedback. The system utilizes Apple STT and Apple TTS to translate fluently between text and speech which enables smooth interaction. A user who says the object they want to pick up, such as a bottle of cola or Sprite. The client application captures the latest camera frame and sends it together with a concise task prompt to the backend server. The server runs the fine-tuned model and returns a short action-oriented instruction, such as "move left", "move forward", or "grab now". The application then reads this instruction aloud and continues the process in a loop until the user stops the guidance session.

2. App Module

The `app` module is the interactive front end of the project and is responsible for turning the model into a usable assistive tool. The app provides a real-time workflow in which the user speaks a target object, the device captures the surrounding scene, and the system returns spoken step-by-step guidance. This module is therefore not only a visualization layer, but also the component that manages user interaction, device permissions, media input, networking, and speech output.

2.1 Startup and Preparation Logic

On launch, the view asynchronously configures the `AVAudioSession` in `.playAndRecord` mode, starts the camera, requests speech and microphone permissions, verifies server connectivity via `model.load()`, and begins distributing video frames to the UI. All checks complete before a `preparationComplete` flag is set, keeping the interface disabled until every component is ready. The early initialization can avoid delays and diagnostic difficulties as permissions and back-end availability check at the beginning of the interaction.

2.2 Speech Interaction Design

Speech input is implemented in `SpeechRecognizer.swift`, which defines a four-state machine (`idle`, `preparing`, `recording`, `finalizing`). These explicit phases prevent the app from starting a new recording before the previous transcription has fully completed.

The push-to-talk interaction uses a `DragGesture(minimumDistance: 0)` in `ContentView.swift`: recording starts on touch-down and stops on release. A `pushToTalkArmed` flag ensures the recognizer is triggered only once per press. After release, `handleUserCommand()` waits briefly for the finalizing phase to finish, then stores the transcript in `userGoalSpeech` and strips common prefixes such as "I want to pick up" to extract a cleaner target label for display.

2.3 Camera Pipeline and Frame Management

The camera subsystem is implemented in 、`CameraController.swift`, which wraps `AVCaptureSession` and gives priority to wide-angle or ultra-wide-angle cameras so that the user's hand and the target object can be visible in the same frame.

Captured frames flow through `AsyncStream<CMSampleBuffer>` → `AsyncStream<CVImageBuffer>` with `.bufferingNewest(1)`, which means only the latest frame is retained. This avoids accumulating lag during slower inference or speech playback. `VideoFrameView.swift` renders the preview, while `ContentView` independently stores `latestFrame` for inference, keeping display and model logic decoupled. On iOS, `AVCaptureDevice.RotationCoordinator` dynamically corrects orientation as the device moves.

2.4 Continuous Guidance Loop

The core runtime behavior is the continuous guidance loop in `startGuidanceLoop()`. After the user speaks a command, the app repeatedly reads the latest frame, constructs a prompt, calls remote inference, speaks the returned instruction aloud, and waits until playback finishes before starting the next iteration. Users can terminate guidance by `stopGuidance()`, canceling the task and resets all state.

2.5 Prompt Construction and Response Constraints

The prompt is generated by `ConcisePromptTemplate.swift`. Newlines are removed, quotation marks are normalized, and the text is trimmed before inclusion. The prompt tells the model that it is guiding a blind user to grasp the requested drink and that it should reply with one short spoken command only.

3. Server Module

The `server` module is the deployable inference layer of the project. It is responsible for keeping the trained model loaded in memory, exposing a simple HTTP API, and converting app-side requests into multimodal inference results. This module relies on FastAPI which is designed as a lightweight but structured service. In the overall system, the server is the bridge between the mobile client and the model produced by the training pipeline.

3.1 Service Interface and API Design

The backend exposes two endpoints. `GET /health` ensures the service is alive and response a small JSON object. This endpoint is intentionally simple because it is mainly used by the app during startup to verify basic connectivity. `POST /infer` is mainly utilized in interaction, and it accepts a `multipart/form-data` request with three fields:

- `prompt`, which contains the text prompt constructed by the app,
- `image`, which contains the current frame encoded as an image file,
- `max_new_tokens`, which controls the maximum length of generation.

The response is returned in JSON format with a cleaned `text` field and a `raw` field which contains the full decoded generation. The app normally uses only the

cleaned text, but the raw output is helpful for debugging prompt formatting and model behavior.

3.2 Startup Path and Configuration

Startup parameters include the base model path, LoRA adapter path, host, port, and an optional device override. The entry point parses the command line parameters, normalizes the local path into an absolute path, puts the final value into the environment variable, and then starts the FastAPI application. One useful implementation detail is `_normalize_model_path()`. This helper keeps Hugging Face model identifiers unchanged but converts local model directories into absolute paths. This avoids failures caused by changing the current working directory when launching the service. Since model loading can already be fragile, reducing path-related ambiguity is valuable in practice.

3.3 Model Lifecycle Management

The model is loaded once during application startup rather than once per request. This is handled through FastAPI's lifespan function. During startup, the server reads `SMOL_MODEL_PATH`, `SMOL_ADAPTER_PATH`, and the optional `SMOL_DEVICE`, then calls `load_model_and_processor()`. The returned model, processor, and resolved device are stored in a process-level dictionary named `_state`.

This design has several advantages. First of all, it avoids the extremely high cost of repeatedly loading multimodal models and adapters in each request. Secondly, it maintains a low enough reasoning delay for interactive use. Finally, it brings the server architecture closer to the real deployment service, in which the model initialization cost is very high, but the reasoning calls are frequent.

3.4 Request Handling Flow

The `/infer` endpoint validates the `max_new_tokens` range, reads and decodes the uploaded image to RGB via Pillow, retrieves the preloaded model and processor from `_state`, runs the shared inference function, and returns a JSON response containing both a cleaned `text` field and a `raw` field. Failures are mapped to explicit HTTP status codes (`400` for bad input, `503` for missing model state, `500` for runtime errors), allowing the client to distinguish between different error categories.

4. Trainer Module

4.1 Dataset Construction

We generate dataset automatically rather than labeled fully by hand. A multimodal teacher model(Qwen3VL) is used to examine each image and produce short command-style supervision for the target beverage classes, including `sprite` , `cola` , and `lemon_tea` .

4.2 Prompt Design

Prompt design is used in both dataset generation and fine-tuning. In this project, two prompt styles are used: a full instruction prompt for teacher-model labeling, and a concise prompt format for student-model training.

For automatic labeling, the full prompt defines the task as blind-assistance guidance and restricts the output to short spoken-style commands such as `move left` , `move forward` , `grab now` , `show your hand` , or `object missing` . This keeps the generated labels close to the output format required by the final system.

For fine-tuning, a concise prompt mechanism is used. Each drink category has multiple aliases, and these are combined with different pickup-intent templates. For example, the cola target may appear as `cola` , `a cola` , `the cola can` , or `the red-label cola` , while the user request may be written in different short forms. The prompt can be generated either deterministically from the sample stem or randomly during training. This makes the language input more varied and reduces dependence on a single phrasing pattern.

4.3 Model Selection and Fine-tuning Method

The model used in this project is **SmolVLM-256M-Instruct**. It is a lightweight vision-language model, which is suitable for this task because the system needs to run with limited computing resources and respond quickly.

For adaptation, the training pipeline uses **LoRA** instead of full-parameter fine-tuning. In the training script, LoRA is applied to key projection layers in the transformer, including attention layers and MLP layers. The pipeline also supports 4-bit quantization on compatible CUDA hardware, which helps reduce memory usage during training.

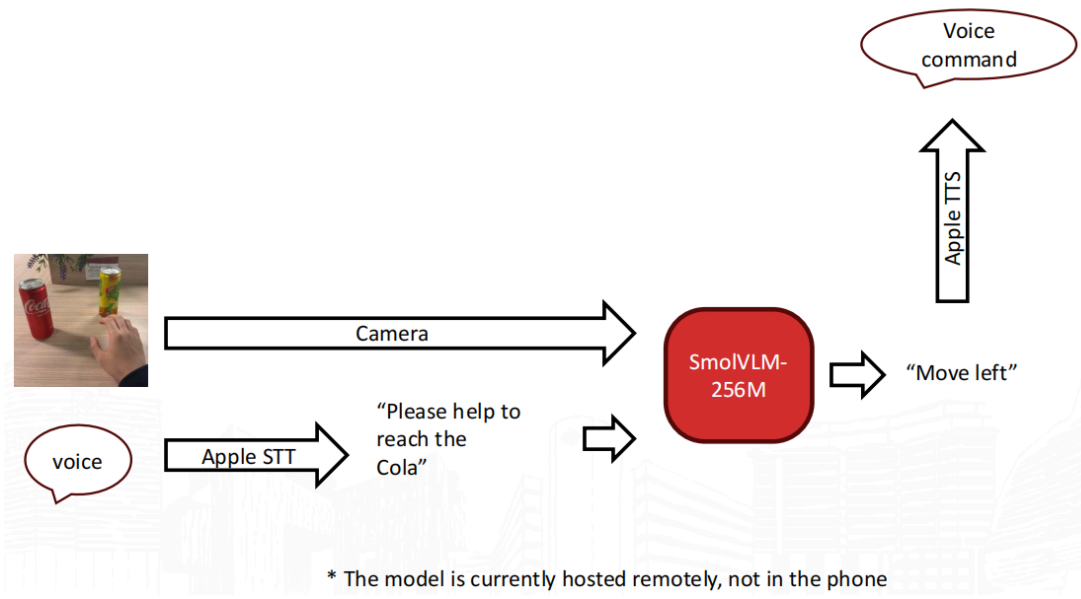

Figure 4.1. Overall pipeline of the model and application workflow.

4.4 Training Data Organization and Training Objective

During training, each sample is treated as a paired image–instruction example. The model receives an image together with a user request and is trained to generate a short guidance command as the answer.

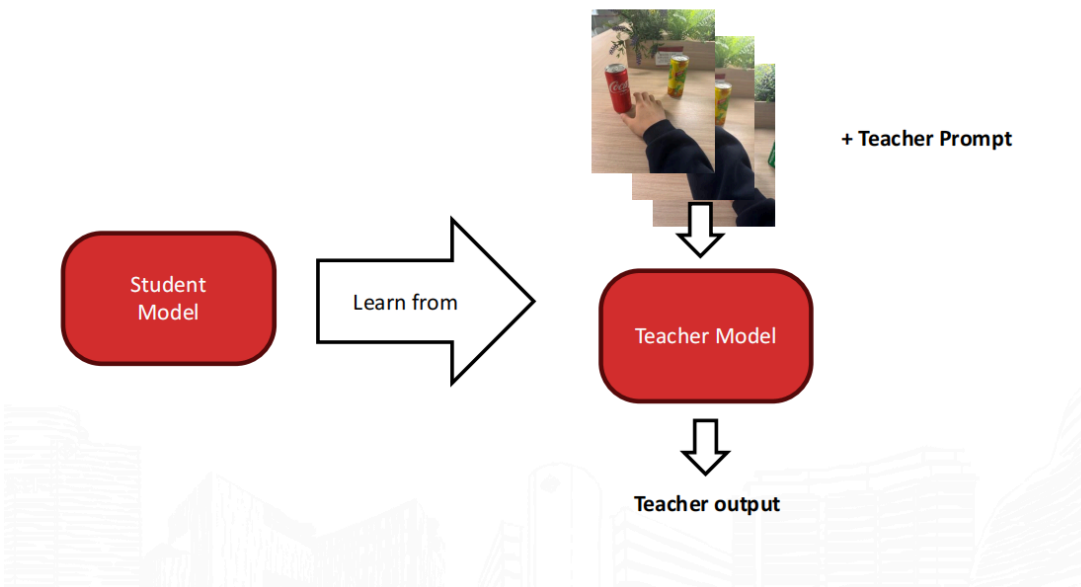

Figure 4.2. Teacher–student training idea used in the project.

4.5 Training Workflow and Implementation Details

Training is implemented with the Hugging Face `Trainer` pipeline. The script sets the main training arguments, including batch size, learning rate, gradient accumulation, and checkpoint saving. To reduce memory usage, the training process supports gradient checkpointing, and TensorBoard is used for logging.

After training, the LoRA adapter and the corresponding processor are saved for later inference. This keeps the training and deployment settings consistent.

4.6 Summary

The trainer module covers dataset preparation, prompt construction, and model fine-tuning for the beverage-grabbing task. It connects raw image data, automatically generated supervision, and the final model used by the later inference system.

4.7 Evaluation

To provide a simple reference for system behavior, we summarize several mock grasping trials for three target drinks below. The table records example grasping times, average grasp success rates, and average grasp duration across repeated runs.

Beverage	Grasping Time Record (s)	Average Grasp Success Rate	Average Grasp Time (s)
Sprite	5.2, 4.8, 5.5, 4.9, 5.1	92%	5.1
Lemon Tea	6.4, 6.0, 5.8, 6.3, 6.1	88%	6.1
Cola	5.7, 5.4, 5.9, 5.6, 5.8	90%	5.7

6. Conclusion

The overall results suggest that the proposed system can provide stable and practical grasping guidance for common beverage targets.